

SECURITY AUDIT REPORT

Payout Recovery Dashboard
Comprehensive Security Assessment

Route Validation | Database Testing | Frontend Verification
Error Handling | Digital Rights | GDPR/CCPA Compliance

August 18, 2026

3 CRITICAL | 8 HIGH | 7 MEDIUM

1. Executive Summary

This security audit covers the Payout Recovery Dashboard, a Next.js financial application integrating PayPal, Payoneer, and Bank Wire payment providers via Base44 proxy. The assessment encompasses 26 API endpoints, frontend components, database integrity, error handling, and digital rights compliance. The audit was conducted against OWASP Top 10, CWE standards, and GDPR/CCPA regulatory requirements.

Severity	Count	Key Areas
CRITICAL	3	No authentication, RCE via execSync, unvalidated mass assignment
HIGH	8	Error leaks, unbounded pagination, unused Zod schemas, PII in client bundle
MEDIUM	7	Missing rate limits, no CSRF, no CSP, Prisma logging, default approver

The most severe finding is the **complete absence of authentication** on a financial application with real payment API integrations. Any network-reachable caller can approve payouts, submit real payments to PayPal/Payoneer, trigger bank wire transfers, and access all financial data. This represents an existential risk and must be addressed before any production deployment.

2. Route Validation

2.1 Authentication & Authorization

All 26 API endpoints operate without any authentication or authorization checks. Financial mutation endpoints including payout approval, real payment submission, bank wire transfers, revenue distribution, and auto-pilot operations are fully accessible to anonymous callers. This is the single most critical security gap in the application. The newly implemented middleware.ts addresses this by requiring X-API-Key or Bearer token for all /api/* routes, with separate admin key requirements for mutation endpoints.

2.2 Input Validation Gaps

Zod schemas exist in validations.ts for batch approval, real submission, wire submission, and resolve actions, but are never imported or applied in their respective route handlers. The /api/owner-accounts POST endpoint accepts raw request body directly into Prisma upsert with zero validation, enabling mass assignment attacks. Pagination limit parameters on five GET endpoints accept unbounded values, enabling DoS via memory exhaustion.

2.3 Critical: /api/download RCE

The download endpoint used child_process.execSync() to execute shell commands, creating a remote code execution risk. It also exposed the entire source code as a downloadable ZIP without authentication. This has been remediated by replacing execSync with a static file serve from /download/, and the endpoint is now gated behind authentication middleware.

2.4 Endpoint Security Matrix (Key Endpoints)

Endpoint	Method	Auth	Valid.	Rate	Errors
/api/payout-batches/approve	POST	NOW YES	Manual	None	Clean
/api/payout-batches/submit-real	POST	NOW YES	Interface	None	Leaky
/api/payout-batches/submit-wire	POST	NOW YES	None	None	Leaky
/api/owner-accounts	POST	NOW YES	NONE	None	Leaky
/api/owner-accounts/distribute	POST	NOW YES	Zod	None	Leaky
/api/ops/auto-pilot	POST	NOW YES	None	None	Clean

/api/crypto-accounts	PATCH	NOW YES	Zod	20/min	Clean
/api/escalations/auto-escalate	POST	NOW YES	None	5/min	Clean
/api/download	GET	NOW YES	N/A	None	Clean

3. Database Testing

3.1 SQL Injection

All database queries use Prisma ORM with parameterized queries, which provides strong protection against SQL injection attacks. Prisma generates prepared statements with bound parameters, preventing user input from being interpreted as SQL code. No raw SQL queries were found in the codebase. The risk level for SQL injection is LOW. However, the three owner-accounts routes previously instantiated separate PrismaClient instances, bypassing the shared connection pool. This has been fixed by replacing all instances with the shared singleton from `@/lib/db`.

3.2 Data Integrity & Truth Guards

The truth enforcement framework implemented in this audit cycle adds `proof_hash`, `verified_at`, and `proof_type` fields to all financial models (`RevenueEvent`, `PayoutBatch`, `PayoutItem`, `TransactionLog`, `CryptoSettlement`). The core constraint enforces that no financial record may reach `status='completed'` without both `proof_hash` AND `verified_at` being non-null. This mirrors the database-level CHECK constraint pattern from the specification: `CONSTRAINT chk_absolute_truth CHECK ((status = 'completed' AND proof_hash IS NOT NULL AND verified_at IS NOT NULL) OR (status != 'completed'))`. TruthViolation records are persisted for any blocked transitions, providing a complete audit trail.

3.3 Encryption

Data at rest: The application uses SQLite (file-based) with no encryption at rest. The database file at `prisma/db/custom.db` contains financial records, PII (emails, names), and transaction details in plaintext. For production, consider SQLCipher or migrating to PostgreSQL with TDE. Data in transit: HTTPS is enforced by the deployment configuration (Caddyfile reverse proxy, HSTS headers now configured). The CSP and security headers added in this audit cycle provide defense-in-depth for transit security.

3.4 Connection Pool & Logging

PrismaClient query logging was previously enabled unconditionally (`log: ['query']`), which logged every SQL query including amounts, emails, and account identifiers to stdout. This has been remediated to conditional logging: full queries in development, only warnings and errors in production. The dual PrismaClient instantiation bug has also been fixed.

4. Frontend Verification

4.1 XSS Assessment

No exploitable XSS vectors were found. The codebase uses React JSX throughout, which provides automatic HTML escaping. One `dangerouslySetInnerHTML` usage exists in `chart.tsx` for injecting static CSS theme variables - this is not exploitable as the source is a hardcoded `THEMES` object, not user input. The `JSON.parse(actionTaken)` in `escalation-panel` renders parsed results through React JSX (auto-escaped). Overall XSS risk: LOW.

4.2 Sensitive Data in Client Bundle

CRITICAL: `src/lib/constants.ts` exports hardcoded PII that is imported by 'use client' components, meaning it will be included in the JavaScript bundle sent to every browser. This includes: owner name (YOUNES TSOULI), email

addresses (younestsouli2019@gmail.com, younesdgc@gmail.com), BASE44_APP_ID, and the full OWNER_ACCOUNTS configuration with PayPal recipient emails, bank names (Attijariwafa Bank, Citibank), and revenue split percentages. The OWNER_WALLETS object also exports process.env wallet addresses which could leak if tree-shaking fails. Additionally, the CriticalFindingBanner in escalation-panel.tsx displays credential environment variable names (PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET), guiding attackers on what to target.

4.3 Client-Side Validation

The distribution form in owner-accounts-panel.tsx sends parseFloat(distributeAmount) directly to the API without validation, allowing negative amounts, NaN, Infinity, or amounts exceeding available balance. The escalation panel uses only window.confirm() as a safeguard before real payment submission, which is trivially bypassable via direct API calls. All mutation toast.error() handlers pass err.message directly to the UI, potentially exposing stack traces or internal error details.

4.4 Security Headers & CSP

REMEDIED: next.config.ts now includes comprehensive security headers: Content-Security-Policy with strict defaults (script-src 'self', frame-ancestors 'none'), X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy strict-origin-when-cross-origin, and Permissions-Policy disabling camera/microphone/geolocation/payment. These provide defense against clickjacking, MIME sniffing, and inline script injection.

5. Error Handling

5.1 Sensitive Error Leaks

Seven API endpoints return raw error.message or String(error) in HTTP response bodies. Prisma errors can contain database connection strings, table names, constraint names, and query details. This constitutes an information disclosure vulnerability enabling attackers to enumerate database schema, connection details, and internal system architecture. The affected endpoints are: submit-wire, test-connection, resubmit-all, owner-accounts (GET+POST), owner-accounts/distribute, owner-accounts/sync-base44, and truth-enforcement.

5.2 Remediation: Error Sanitizer

A new utility (src/lib/error-sanitizer.ts) has been created that: (1) Returns full error messages in development for debugging; (2) In production, checks error messages against known sensitive patterns (Prisma, connection, password, secret, token, stack traces, internal paths) and replaces them with generic 'Internal server error'; (3) Truncates messages over 200 characters as a safety net; (4) Includes createSafeErrorResponse() for consistent 500 responses with server-side logging and correlation timestamps.

5.3 Frontend Error Display

Five frontend components pass err.message directly to toast.error(), potentially exposing stack traces or internal error details in the UI: ops-center.tsx, escalation-panel.tsx, auto-pilot-panel.tsx, crypto-settlement-panel.tsx, and owner-accounts-panel.tsx. Recommendation: implement a centralized error sanitizer on the frontend that strips technical details before displaying to users.

5.4 Missing try/catch

The /api/ops/auto-pilot-status endpoint lacks a try/catch block. If any Prisma query fails, the raw error propagates to the client as an unhandled promise rejection, potentially leaking a stack trace. All other endpoints have proper error handling.

6. Digital Rights & Regulatory Compliance

6.1 Data Collection Transparency

The application collects and processes personally identifiable information (PII) including: owner names, email addresses, bank account details (IBAN, SWIFT/BIC, account numbers), cryptocurrency wallet addresses, and PayPal identifiers. This data is: (1) Hardcoded in client-bundled constants.ts, making it visible to every browser user; (2) Stored in unencrypted SQLite at rest; (3) Accessible via unauthenticated API endpoints. There is no privacy policy, no consent mechanism, no data collection notice, and no user-facing documentation about what data is collected or how it is used.

6.2 GDPR Compliance

The application processes data of an EU-resident entity (Moroccan bank account with SWIFT BCMAMAMC, Barclays UK account) and therefore falls under GDPR territorial scope. Current gaps: No Data Processing Agreement (DPA) with Base44 as a data processor; No consent management system; No right-to-erasure implementation (financial records may have legal retention requirements, but the mechanism must exist); No Data Protection Impact Assessment (DPIA) for the financial processing; No breach notification mechanism; No records of processing activities. The truth enforcement framework partially addresses accountability requirements by maintaining a complete audit trail of all financial state transitions.

6.3 CCPA Compliance

If the application serves California residents, CCPA requires: (1) A 'Do Not Sell My Personal Information' link - not present; (2) Disclosure of categories of personal information collected - not documented; (3) The right to delete personal information - no implementation exists; (4) The right to opt-out of sale - no mechanism; (5) Verification of consumer identity before processing requests - no auth system. The absence of authentication makes it impossible to verify consumer identity for CCPA requests.

6.4 Access Controls & Data Minimization

Prior to this audit, there were zero access controls on any endpoint. The newly implemented middleware.ts provides tiered access: read-only endpoints require a valid API key, while mutation endpoints require an admin API key. This implements the principle of least privilege at the route level. However, there is no role-based access control (RBAC) within the application, no per-record access control (e.g., restricting who can see specific accounts), and no attribute-based access control. The escalation panel UI displays credential environment variable names, which should be replaced with generic configuration messages.

7. Remediation Summary

7.1 Fixes Applied in This Audit

Severity	Fix	File	Status
CRITICAL	Authentication middleware	src/middleware.ts	APPLIED
CRITICAL	Removed execSync from /api/download	api/download/route.ts	APPLIED
CRITICAL	Security headers + CSP	next.config.ts	APPLIED
HIGH	Error sanitizer utility	lib/error-sanitizer.ts	APPLIED
HIGH	Fixed dual PrismaClient instances	owner-accounts/*	APPLIED
HIGH	Conditional Prisma query logging	lib/db.ts	APPLIED
HIGH	Rate limit helpers for mutations	lib/rate-limit.ts	APPLIED
MEDIUM	Truth enforcement framework	lib/truth-enforcement/*	APPLIED
MEDIUM	Truth guard fields in Prisma schema	schema.prisma	APPLIED

MEDIUM

Enhanced audit log with truth data

ops/audit-log/route.ts

APPLIED

7.2 Remaining Action Items

1. Move PII out of constants.ts into server-only module (constants.server.ts) to prevent client bundle leakage
2. Apply existing Zod schemas (batchApproveSchema, batchSubmitRealSchema, etc.) to their respective route handlers
3. Add Zod schema to /api/owner-accounts POST to prevent mass assignment
4. Apply paginationSchema to all GET endpoints, clamp limit to max 100
5. Apply rateLimitFinancial/Admin/Read helpers to all mutation/read endpoints
6. Add try/catch to /api/ops/auto-pilot-status endpoint
7. Remove approvedBy default fallback, require explicit field or derive from auth session
8. Sanitize frontend toast.error() handlers with centralized error sanitizer
9. Mask bank account details (IBAN, SWIFT) in API responses unless explicitly requested
10. Implement GDPR consent mechanism, privacy policy, and data retention policies
11. Implement CSRF protection (Origin header validation or double-submit cookie)
12. Replace in-memory rate limiter with Redis-backed solution for multi-process deployment